

Chapter 2

Advanced Evasion Attacks

PCC190 – Secure Machine Learning
Prof. Rodrigo C. P. Silva – DECOM/UFOP

October 2, 2026

Contents

Setting and notation	1
1 Gradient-Based Attacks: FGSM, BIM and PGD	2
1.1 Fast Gradient Sign Method (FGSM)	2
1.2 Basic Iterative Method (BIM)	2
1.3 Projected Gradient Descent (PGD)	2
2 Optimization-Based Attacks: Carlini and Wagner	3
3 Score-Based Attacks	4
3.1 Zeroth-order gradient estimation	4
3.2 Gradient-free random search	4
4 Decision-Based Attacks	4
5 Transferability of Adversarial Examples	5
6 Attacking Ensemble Models	5
7 Hands-On: Implementing Evasion Attacks	6
7.1 FGSM and PGD	6
7.2 Carlini–Wagner ℓ_2 (single c , no binary search)	6
7.3 A score-based attack: NES gradient estimate inside PGD	7
7.4 Transfer from an ensemble of surrogates	7
7.5 Suggested experiments	7
Exercises	8

Setting and notation

We keep the notation of Chapter 1: classifier f with logits $Z(\mathbf{x}) \in \mathbb{R}^K$, probabilities $F(\mathbf{x})$, loss $\mathcal{L}(\mathbf{x}, y)$ (we drop θ since the model is fixed at inference time), threat set $\mathcal{B}_p(\mathbf{x}, \epsilon)$ and valid input domain $\mathcal{X} = [0, 1]^d$. All attacks in this chapter solve, exactly or approximately, one of two problems:

$$(\text{budget}) \quad \max_{\mathbf{x}' \in \mathcal{B}_p(\mathbf{x}, \epsilon) \cap \mathcal{X}} \mathcal{L}(\mathbf{x}', y) \quad (\text{min-norm}) \quad \min_{\delta} \|\delta\|_p \text{ s.t. } f(\mathbf{x} + \delta) \neq y, \mathbf{x} + \delta \in \mathcal{X}.$$

What changes from one family to the next is the information the attacker has about f : gradients (white-box), scores, labels only, or nothing about the target at all (transfer). We denote by $\Pi_S(\cdot)$ the Euclidean projection onto a set S , and by $\text{clip}_{[0,1]}$ the elementwise projection onto the box.

1 Gradient-Based Attacks: FGSM, BIM and PGD

1.1 Fast Gradient Sign Method (FGSM)

Linearize the loss around $\mathbf{x}$: $\mathcal{L}(\mathbf{x} + \delta, y) \approx \mathcal{L}(\mathbf{x}, y) + \delta^\top \mathbf{g}$, with $\mathbf{g} = \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, y)$. Maximizing this linear model over $\|\delta\|_p \leq \epsilon$ is a closed-form problem whose solution is given by the dual norm. For $p = \infty$, the maximizer is $\delta = \epsilon \text{sign}(\mathbf{g})$, and the attack is (Goodfellow et al., 2015)

$$\mathbf{x}' = \text{clip}_{[0,1]}(\mathbf{x} + \epsilon \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, y))). \quad (1)$$

For $p = 2$ the same reasoning gives the *fast gradient method* (FGM), $\delta = \epsilon \mathbf{g} / \|\mathbf{g}\|_2$. The targeted version descends the loss of the target class: $\mathbf{x}' = \mathbf{x} - \epsilon \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, t))$.

Proposition 1.1 (Steepest ascent direction). *For any $\mathbf{g} \in \mathbb{R}^d$ and $p \geq 1$, $\max_{\|\delta\|_p \leq \epsilon} \delta^\top \mathbf{g} = \epsilon \|\mathbf{g}\|_q$, with $1/p + 1/q = 1$. For $p = \infty$ (so $q = 1$) the maximum is attained at $\delta = \epsilon \text{sign}(\mathbf{g})$; for $p = 2$ at $\delta = \epsilon \mathbf{g} / \|\mathbf{g}\|_2$.*

Analysis. FGSM costs one forward and one backward pass. It is exactly optimal only if the loss is linear in the ball, which it is not for deep networks. Its value today is as a fast baseline and as an inexpensive generator for adversarial training. It is a *weak* attack for evaluation: a model can resist FGSM while being completely broken by iterative attacks. In fact, models trained only on FGSM examples are known to exhibit *label leaking* and a form of gradient masking where the loss surface becomes highly curved near the data, making the single linear step uninformative (Tramèr et al., 2018).

1.2 Basic Iterative Method (BIM)

Kurakin et al. (2017) proposed taking many small FGSM steps of size $\alpha < \epsilon$ and clipping back into the ball after each step:

$$\mathbf{x}^{(0)} = \mathbf{x}, \quad \mathbf{x}^{(k+1)} = \text{clip}_{\mathcal{B}_\infty(\mathbf{x}, \epsilon) \cap [0,1]^d}(\mathbf{x}^{(k)} + \alpha \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^{(k)}, y))). \quad (2)$$

A common heuristic is $T = \lceil \min(\epsilon + 4, 1.25\epsilon) \rceil$ steps for ϵ in 0-255 units with $\alpha = 1/255$, so that the iterate can reach the boundary of the ball.

1.3 Projected Gradient Descent (PGD)

Madry et al. (2018) framed the iterative attack as projected (steepest) ascent on the constrained problem, with two practical additions: a *random start* inside the ball and, often, *multiple restarts*.

Algorithm 1 PGD attack, ℓ_∞ , untargeted

Require: $\mathbf{x}, y$, budget ϵ , step size α , iterations T , restarts R

```

1:  $\mathbf{x}'_{\text{best}} \leftarrow \mathbf{x}$ 
2: for  $r = 1, \dots, R$  do
3:    $\mathbf{x}^{(0)} \leftarrow \text{clip}_{[0,1]}(\mathbf{x} + \mathbf{u})$ ,  $\mathbf{u} \sim \mathcal{U}[-\epsilon, \epsilon]^d$ 
4:   for  $k = 0, \dots, T - 1$  do
5:      $\mathbf{g} \leftarrow \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^{(k)}, y)$ 
6:      $\mathbf{x}^{(k+1)} \leftarrow \Pi_{\mathcal{B}_\infty(\mathbf{x}, \epsilon) \cap [0,1]^d}(\mathbf{x}^{(k)} + \alpha \text{sign}(\mathbf{g}))$ 
7:   end for
8:   keep  $\mathbf{x}^{(T)}$  as  $\mathbf{x}'_{\text{best}}$  if it misclassifies (or has higher loss)
9: end for
10: return  $\mathbf{x}'_{\text{best}}$ 
```

For the ℓ_∞ ball intersected with the box, the projection is separable and reduces to elementwise clipping: $\Pi(\mathbf{z})_i = \min(\max(z_i, x_i - \epsilon, 0), x_i + \epsilon, 1)$. For ℓ_2 , the step is $\alpha \mathbf{g} / \|\mathbf{g}\|_2$ and the projection onto the ball rescales δ to norm ϵ if it exceeds it (the exact projection onto the intersection with the box is not a simple composition, but clipping after rescaling is the usual approximation).

Why PGD became the reference first-order attack. Madry et al. (2018) observed empirically that, from many random starts, PGD finds local maxima of the loss with remarkably concentrated values. This suggests that, for first-order methods, PGD is close to the best one can do, which motivated using PGD both to evaluate models and to train them. This is an empirical observation, not a theorem: the inner problem is nonconcave and PGD offers no global guarantee.

Practical analysis.

- **Step size and iterations.** Too large an α oscillates at the corners of the ball; too small an α with few iterations never reaches the boundary. A typical CIFAR-10 setting is $\epsilon = 8/255$, $\alpha = 2/255$, $T \in [20, 100]$.
- **Loss choice.** Cross-entropy saturates when the model is very confident, giving tiny gradients. Margin losses on logits, such as $\max_{j \neq y} Z_j(\mathbf{x}) - Z_y(\mathbf{x})$ (Section 2), or the difference-of-logits-ratio loss, are often stronger.
- **Fixed schedules are brittle.** APGD (Croce and Hein, 2020) adapts the step size and adds momentum, removing the hyperparameter sensitivity of PGD. Together with complementary attacks, it forms AutoAttack, the current standard for reporting ℓ_∞ and ℓ_2 robustness (Chapter 6).

Reading PGD results

A PGD robust accuracy is an *upper bound* on true robust accuracy. Sanity checks: accuracy should decrease monotonically with ϵ and reach zero for large ϵ ; more iterations and restarts should not increase accuracy; PGD should beat FGSM; white-box should beat black-box. Violations indicate gradient masking or a bug (Carlini et al., 2019).

2 Optimization-Based Attacks: Carlini and Wagner

Carlini and Wagner (2017) (C&W) attack the minimum-norm problem directly. Two obstacles must be removed: the constraint $f(\mathbf{x} + \delta) = t$ is discrete, and the box constraint $\mathbf{x} + \delta \in [0, 1]^d$ is awkward for gradient methods.

From a hard constraint to a penalty. Replace the classification constraint by a function h such that $f(\mathbf{x} + \delta) = t$ holds whenever $h(\mathbf{x} + \delta) \leq 0$. After comparing several candidates, C&W recommend the logit margin

$$h(\mathbf{x}') = \max \left(\max_{j \neq t} Z_j(\mathbf{x}') - Z_t(\mathbf{x}'), -\kappa \right), \quad (3)$$

where $\kappa \geq 0$ is a *confidence* parameter: the attack keeps pushing until the target logit exceeds every other logit by at least κ . For the untargeted version, use $\max(Z_y - \max_{j \neq y} Z_j, -\kappa)$. Working with logits instead of softmax probabilities avoids saturation.

Removing the box constraint. Change variables to $\mathbf{w} \in \mathbb{R}^d$ with

$$\mathbf{x}' = \frac{1}{2} (\tanh(\mathbf{w}) + 1),$$

so that $\mathbf{x}' \in [0, 1]^d$ automatically and the problem becomes unconstrained in $\mathbf{w}$.

The ℓ_2 attack. Combining both,

$$\min_{\mathbf{w}} \left\| \frac{1}{2} (\tanh(\mathbf{w}) + 1) - \mathbf{x} \right\|_2^2 + c \cdot h\left(\frac{1}{2} (\tanh(\mathbf{w}) + 1)\right), \quad (4)$$

solved with Adam. The constant $c > 0$ trades off distortion against success; it is chosen per input by *binary search*: increase c when the attack fails, decrease it when it succeeds, and keep the smallest successful perturbation.

ℓ_0 and ℓ_∞ variants. The ℓ_∞ norm is non-differentiable and gradient descent on it oscillates, so C&W replace it by the penalty $\sum_i \max(|\delta_i| - \tau, 0)$ and shrink τ over iterations. The ℓ_0 attack repeatedly runs the ℓ_2 attack and freezes the pixels that contributed least, shrinking the set of modifiable features.

Analysis.

- C&W produces much smaller perturbations than FGSM/PGD at a fixed success rate and was historically decisive in breaking defenses such as defensive distillation, which shrank softmax gradients but left logits informative.
- It is expensive: thousands of gradient steps per input including the binary search.
- It reports a *distance*, not a success rate at fixed ϵ . To compare with PGD, count the fraction of inputs whose minimal C&W perturbation is below ϵ .
- The margin loss (Eq. 3) is widely reused inside PGD (“PGD-CW”), often outperforming cross-entropy.

Remark 2.1. DeepFool (Moosavi-Dezfooli et al., 2016) is a cheaper minimum-norm alternative: at each step it linearizes all class boundaries and jumps to the nearest one, which for a linear classifier recovers the exact minimal ℓ_2 perturbation (compare Exercise 2 of Chapter 1).

3 Score-Based Attacks

Now suppose the attacker can query $F(\mathbf{x})$ (or the top- k scores) but has no gradients. The loss $\mathcal{L}(\mathbf{x}, y)$ is still computable from scores, so the budget problem can still be attacked, provided we can *estimate* the gradient or avoid it.

3.1 Zeroth-order gradient estimation

Coordinate-wise finite differences. ZOO (Chen et al., 2017) estimates partial derivatives with symmetric differences,

$$\hat{g}_i = \frac{\mathcal{L}(\mathbf{x} + h\mathbf{e}_i, y) - \mathcal{L}(\mathbf{x} - h\mathbf{e}_i, y)}{2h},$$

updating a random batch of coordinates per iteration (with a C&W-style objective). An exact gradient needs $2d$ queries, which is prohibitive for images; ZOO mitigates this by attacking in a downsampled space and prioritizing important coordinates.

Random-direction estimators (NES). Ilyas et al. (2018) use natural evolution strategies. They sample n directions $\mathbf{u}_i \sim \mathcal{N}(0, I)$ and estimate

$$\hat{\mathbf{g}} = \frac{1}{2n\sigma} \sum_{i=1}^n [\mathcal{L}(\mathbf{x} + \sigma\mathbf{u}_i, y) - \mathcal{L}(\mathbf{x} - \sigma\mathbf{u}_i, y)] \mathbf{u}_i, \quad (5)$$

which is an unbiased estimate of the gradient of a Gaussian-smoothed loss. Plugging $\hat{\mathbf{g}}$ into PGD gives an effective attack with far fewer than d queries per step. The same paper handles the *partial-information* (top- k only) and *label-only* settings. SPSA (Uesato et al., 2018) is a closely related estimator using Rademacher directions; it is also a standard tool to test for gradient masking, since it does not rely on the model’s backpropagated gradients.

3.2 Gradient-free random search

Square Attack. Andriushchenko et al. (2020) dispense with gradient estimation. At each iteration a random square window is chosen and the perturbation inside it is resampled at the vertices of the ℓ_∞ ball (values $\pm\epsilon$); the change is accepted only if the margin loss decreases. Window size shrinks over time. Despite its simplicity, Square Attack is among the most query-efficient score-based attacks and, because it never touches gradients, is immune to gradient masking. It is part of AutoAttack for that reason.

Analysis. Score-based attacks reveal that hiding gradients is not a defense: if scores are exposed, gradients can be approximated. Their cost is measured in queries, typically hundreds to a few thousand per image for untargeted ℓ_∞ attacks on CIFAR-scale models, more for targeted attacks and larger inputs. Defenses that only return labels, add noise to scores, or rate-limit queries raise the cost but do not eliminate the threat.

4 Decision-Based Attacks

In the hardest query setting the attacker sees only $f(\mathbf{x})$. The loss is now a step function, and its finite differences are zero almost everywhere. Decision-based attacks therefore work with the geometry of the decision boundary directly, and naturally solve the *min-norm* problem.

Boundary Attack. Brendel et al. (2018) start from a point that is already adversarial (for untargeted attacks, random noise that is misclassified; for targeted attacks, an image of the target class) and perform a random walk *along* the boundary that progressively reduces the distance to $\mathbf{x}$. Each step combines an orthogonal random perturbation (exploring along the boundary) and a small step toward $\mathbf{x}$, and is accepted only if the point remains adversarial. Step sizes are adapted from the acceptance rate. It needs only labels but can require tens of thousands of queries.

HopSkipJump. Chen et al. (2020) improve query efficiency by estimating the gradient direction *at the boundary* from binary feedback. At a boundary point $\mathbf{x}'_b$, define $\phi(\mathbf{z}) = +1$ if $\mathbf{z}$ is adversarial and -1 otherwise. Then

$$\hat{\mathbf{V}} = \frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}'_b + \sigma\mathbf{u}_i) \mathbf{u}_i$$

points, in expectation and for small σ , along the boundary normal. Each iteration: (i) estimate the normal, (ii) take a geometric step along it, (iii) binary-search back to the boundary on the segment toward $\mathbf{x}$. It supports both ℓ_2 and ℓ_∞ and converges with far fewer queries than the Boundary Attack.

Information versus cost

White-box	exact gradients	tens of forward/backward passes
Score-based	loss values	10^2 – 10^4 queries
Decision-based	labels	10^3 – 10^5 queries
Transfer	nothing from target	zero queries, lower success
Less information means higher cost and, usually, larger perturbations, but almost never complete protection.		

5 Transferability of Adversarial Examples

Szegedy et al. (2014) already noted that adversarial examples crafted for one network often fool another network trained on different data or with a different architecture. Papernot et al. (2017) turned this into a practical black-box attack: train a *substitute* model by querying the target on synthetic inputs (using Jacobian-based dataset augmentation), attack the substitute with white-box methods, and submit the result to the target.

Why transfer works. Two complementary explanations:

- **Shared subspaces.** Tramèr et al. (2017) estimate that adversarial perturbations span a subspace of considerable dimension around each input, and that the decision boundaries of different models are close and aligned within it, so perturbations crossing one boundary tend to cross the other.
- **Shared non-robust features.** If models trained on the same distribution learn the same brittle but predictive features (Ilyas et al., 2019), a perturbation that manipulates those features affects all of them.

What makes examples transfer better. Strong white-box optimization tends to *overfit* to the surrogate: highly specific perturbations exploit idiosyncrasies of one model. Empirically:

- Untargeted examples transfer far better than targeted ones (Liu et al., 2017).
- Higher-confidence examples (larger κ in C&W) transfer better than minimal ones.
- *Momentum* stabilizes the update direction and reduces overfitting to the surrogate (MI-FGSM, Dong et al., 2018):

$$\mathbf{m}^{(k+1)} = \mu \mathbf{m}^{(k)} + \frac{\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^{(k)}, y)}{\|\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^{(k)}, y)\|_1}, \quad \mathbf{x}^{(k+1)} = \Pi(\mathbf{x}^{(k)} + \alpha \text{sign}(\mathbf{m}^{(k+1)})).$$

- *Input diversity*: applying random resizing and padding before each gradient computation acts like data augmentation for the attack (Xie et al., 2019).
- Attacking an *ensemble* of surrogates (next section).

Transfer and evaluation

Transfer success depends heavily on surrogate choice; low transfer success is not evidence of robustness. Conversely, if a transfer attack beats your white-box attack on a defended model, the white-box attack is failing, which is a classic sign of gradient masking.

6 Attacking Ensemble Models

Ensembles appear in this chapter in two roles: as targets that claim robustness, and as surrogates that improve transfer.

Ensembles as surrogates. Liu et al. (2017) showed that optimizing a perturbation against several models simultaneously yields examples that transfer much better, including targeted examples, to an unseen model. For models $f_1, \dots, f_M$ with weights $w_m \geq 0$, $\sum_m w_m = 1$, the common fusion choices are

$$\begin{aligned} \text{logit fusion: } Z_{\text{ens}}(\mathbf{x}) &= \sum_m w_m Z_m(\mathbf{x}), \\ \text{probability fusion: } F_{\text{ens}}(\mathbf{x}) &= \sum_m w_m F_m(\mathbf{x}), \\ \text{loss fusion: } \mathcal{L}_{\text{ens}}(\mathbf{x}, y) &= \sum_m w_m \mathcal{L}_m(\mathbf{x}, y). \end{aligned}$$

Logit fusion is usually the most effective for transfer. The intuition: a direction that fools many diverse models is more likely to exploit features shared by all models, including the unseen target.

Ensembles as defenses. An ensemble is a single differentiable function. If the defender averages logits or probabilities, a white-box attacker simply attacks the averaged output with PGD, exactly as for a single model. Problems arise only with *non-differentiable* aggregation (majority vote) or with *randomized* selection of members. In those cases:

- Replace the vote with a differentiable surrogate (average of logits or probabilities) for gradient computation; the resulting example usually fools the vote as well.
- For random member selection, optimize the *expected* loss, $\mathbb{E}_m[\mathcal{L}_m(\mathbf{x}', y)]$, i.e., Expectation over Transformation (Athalye et al., 2018).
- For ensembles of detectors or heterogeneous defenses, attack all components jointly with a combined loss. He et al. (2017) showed that combining several weak defenses does not yield a strong one under adaptive attack.

Ensemble adversarial training (Tramèr et al., 2018) is a different idea: augmenting training with adversarial examples transferred from other, fixed models. It improves robustness to *transfer* attacks specifically, not to white-box attacks on the defended model.

7 Hands-On: Implementing Evasion Attacks

The following PyTorch code implements the attacks of this chapter in their simplest form. It assumes a model in `eval()` mode that receives images in $[0, 1]$ (place any normalization inside the model), and batches $\mathbf{x}$, y . Any CIFAR-10 classifier will do; pretrained robust and non-robust models are available in RobustBench.

7.1 FGSM and PGD

```
import torch
import torch.nn.functional as F

def fgsm(model, x, y, eps):
    x = x.clone().detach().requires_grad_(True)
    loss = F.cross_entropy(model(x), y)
    grad, = torch.autograd.grad(loss, x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

def margin_loss(logits, y):
    # max_{j != y} Z_j - Z_y (positive => misclassified)
    true = logits.gather(1, y[:, None]).squeeze(1)
    other = logits.clone()
    other.scatter_(1, y[:, None], -float("inf"))
    return other.max(1).values - true

def pgd_linf(model, x, y, eps, alpha, steps, restarts=1, loss="ce"):
    best = x.clone()
    fooled = torch.zeros(len(x), dtype=torch.bool, device=x.device)
    for _ in range(restarts):
        delta = torch.empty_like(x).uniform_(-eps, eps)
        xa = (x + delta).clamp(0, 1).detach()
        for _ in range(steps):
            xa.requires_grad_(True)
            logits = model(xa)
            l = F.cross_entropy(logits, y) if loss == "ce" \
                else margin_loss(logits, y).sum()
            grad, = torch.autograd.grad(l, xa)
            xa = xa.detach() + alpha * grad.sign()
            xa = torch.min(torch.max(xa, x - eps), x + eps).clamp(0, 1)
        with torch.no_grad():
            succ = model(xa).argmax(1) != y
            new = succ & ~fooled
            best[new] = xa[new]
            fooled |= succ
    return best, fooled
```

7.2 Carlini–Wagner ℓ_2 (single c , no binary search)

```
def cw_l2(model, x, y, c=1.0, kappa=0.0, steps=200, lr=1e-2):
    # change of variables: x' = (tanh(w) + 1) / 2
    w = torch.atanh((x * 2 - 1).clamp(-1 + 1e-6, 1 - 1e-6)).detach()
    w.requires_grad_(True)
    opt = torch.optim.Adam([w], lr=lr)
    for _ in range(steps):
        xa = (torch.tanh(w) + 1) / 2
        logits = model(xa)
        true = logits.gather(1, y[:, None]).squeeze(1)
        other = logits.clone()
        other.scatter_(1, y[:, None], -float("inf"))
        h = torch.clamp(true - other.max(1).values, min=-kappa)
        dist = ((xa - x) ** 2).flatten(1).sum(1)
        loss = (dist + c * h).sum()
        opt.zero_grad(); loss.backward(); opt.step()
    return ((torch.tanh(w) + 1) / 2).detach()
```

7.3 A score-based attack: NES gradient estimate inside PGD

```
@torch.no_grad()
def nes_grad(model, x, y, sigma=1e-3, n=50):
    g = torch.zeros_like(x)
    for _ in range(n // 2):
        u = torch.randn_like(x)
        lp = F.cross_entropy(model(x + sigma * u), y, reduction="none")
        lm = F.cross_entropy(model(x - sigma * u), y, reduction="none")
        g += (lp - lm).view(-1, *1] * (x.dim() - 1)) * u
    return g / (n * sigma) # n queries per estimate

def nes_pgd(model, x, y, eps, alpha, steps, **kw):
    xa = x.clone()
    for _ in range(steps):
        g = nes_grad(model, xa, y, **kw)
        xa = xa + alpha * g.sign()
        xa = torch.min(torch.max(xa, x - eps), x + eps).clamp(0, 1)
    return xa
```

7.4 Transfer from an ensemble of surrogates

```
class LogitEnsemble(torch.nn.Module):
    def __init__(self, models, weights=None):
        super().__init__()
        self.models = torch.nn.ModuleList(models)
        self.w = weights or [1 / len(models)] * len(models)
    def forward(self, x):
        return sum(w * m(x) for w, m in zip(self.w, self.models))

# craft on surrogates, evaluate on the held-out target
surrogate = LogitEnsemble([resnet18, vgg16, densenet121]).eval()
x_adv, _ = pgd_linf(surrogate, x, y, eps=8/255, alpha=2/255, steps=20)
transfer_rate = (target(x_adv).argmax(1) != y).float().mean()
```

7.5 Suggested experiments

1. On a standard (non-robust) CIFAR-10 model, plot accuracy versus $\epsilon \in \{0, 1, 2, 4, 8\}/255$ for FGSM, PGD-20 with cross-entropy and PGD-20 with the margin loss. Repeat on a robust model from RobustBench. Which attack is strongest, and does the ranking change?
2. For PGD, vary α and the number of steps with fixed $\epsilon = 8/255$. Identify a configuration that underestimates attack strength and explain why.
3. Run C&W ℓ_2 with binary search over c on 100 test images. Report the median minimal ℓ_2 distance and compare the fraction below $\epsilon_2 = 0.5$ with PGD- ℓ_2 at that budget.
4. Implement NES-PGD and plot success rate versus number of queries. How many queries are needed to match 90% of white-box PGD success?
5. Craft examples on a single surrogate and on a three-model logit ensemble. Measure transfer to a held-out architecture, with and without momentum (MI-FGSM). Report untargeted and targeted transfer rates.

Implementation pitfalls

- Keep the model in `eval()` mode; batch normalization in training mode changes outputs per batch.
- Put normalization inside the model so that ϵ is expressed in pixel space.
- Evaluate only on initially correctly classified points, or report clean accuracy alongside robust accuracy.
- Check that $\|\mathbf{x}' - \mathbf{x}\|_\infty \leq \epsilon$ and $\mathbf{x}' \in [0, 1]^d$ after the attack. Bugs here silently inflate success rates.

Exercises

1. Prove the steepest ascent proposition using Hölder’s inequality, and derive the FGM update for ℓ_1 . Why is the ℓ_1 steepest ascent step a poor attack in practice?
2. Show that for $p = \infty$, Euclidean projection onto $\mathcal{B}_\infty(\mathbf{x}, \epsilon) \cap [0, 1]^d$ is elementwise clipping. Why does this fail for ℓ_2 ?
3. Explain why cross-entropy may yield vanishing gradients for highly confident predictions, while the margin loss of Eq. (3) does not.
4. Show that Eq. (5) is an unbiased estimator of $\nabla_{\mathbf{x}} \mathbb{E}_{\mathbf{u} \sim \mathcal{N}(0, I)} [\mathcal{L}(\mathbf{x} + \sigma \mathbf{u}, y)]$. (Hint: Stein’s lemma.)
5. A defender uses a majority vote over five independently trained networks and reports that PGD on each individual member fails to fool the vote. Describe a correct adaptive attack.

References

- Andriushchenko, M., Croce, F., Flammarion, N., and Hein, M. (2020). Square attack: A query-efficient black-box adversarial attack via random search. In *European Conference on Computer Vision (ECCV)*, pages 484–501.
- Athalye, A., Engstrom, L., Ilyas, A., and Kwok, K. (2018). Synthesizing robust adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 284–293.
- Brendel, W., Rauber, J., and Bethge, M. (2018). Decision-based adversarial attacks: Reliable attacks against black-box machine learning models. In *International Conference on Learning Representations (ICLR)*.
- Carlini, N., Athalye, A., Papernot, N., Brendel, W., Rauber, J., Tsipras, D., Goodfellow, I., Madry, A., and Kurakin, A. (2019). On evaluating adversarial robustness. *arXiv preprint arXiv:1902.06705*.
- Carlini, N. and Wagner, D. (2017). Towards evaluating the robustness of neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, pages 39–57.
- Chen, J., Jordan, M. I., and Wainwright, M. J. (2020). HopSkipJumpAttack: A query-efficient decision-based attack. In *IEEE Symposium on Security and Privacy (S&P)*, pages 1277–1294.
- Chen, P.-Y., Zhang, H., Sharma, Y., Yi, J., and Hsieh, C.-J. (2017). ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 15–26.
- Croce, F. and Hein, M. (2020). Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 2206–2216.
- Dong, Y., Liao, F., Pang, T., Su, H., Zhu, J., Hu, X., and Li, J. (2018). Boosting adversarial attacks with momentum. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9185–9193.
- Goodfellow, I. J., Shlens, J., and Szegedy, C. (2015). Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*.
- He, W., Wei, J., Chen, X., Carlini, N., and Song, D. (2017). Adversarial example defenses: Ensembles of weak defenses are not strong. In *11th USENIX Workshop on Offensive Technologies (WOOT)*.
- Ilyas, A., Engstrom, L., Athalye, A., and Lin, J. (2018). Black-box adversarial attacks with limited queries and information. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 2137–2146.
- Ilyas, A., Santurkar, S., Tsipras, D., Engstrom, L., Tran, B., and Madry, A. (2019). Adversarial examples are not bugs, they are features. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Kurakin, A., Goodfellow, I., and Bengio, S. (2017). Adversarial examples in the physical world. In *ICLR Workshop Track*.

- Liu, Y., Chen, X., Liu, C., and Song, D. (2017). Delving into transferable adversarial examples and black-box attacks. *International Conference on Learning Representations (ICLR)*.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., and Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*.
- Moosavi-Dezfooli, S.-M., Fawzi, A., and Frossard, P. (2016). DeepFool: A simple and accurate method to fool deep neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2574–2582.
- Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z. B., and Swami, A. (2017). Practical black-box attacks against machine learning. In *Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIACCS)*, pages 506–519.
- Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., and Fergus, R. (2014). Intriguing properties of neural networks. In *International Conference on Learning Representations (ICLR)*.
- Tramèr, F., Kurakin, A., Papernot, N., Goodfellow, I., Boneh, D., and McDaniel, P. (2018). Ensemble adversarial training: Attacks and defenses. In *International Conference on Learning Representations (ICLR)*.
- Tramèr, F., Papernot, N., Goodfellow, I., Boneh, D., and McDaniel, P. (2017). The space of transferable adversarial examples. *arXiv preprint arXiv:1704.03453*.
- Uesato, J., O’Donoghue, B., van den Oord, A., and Kohli, P. (2018). Adversarial risk and the dangers of evaluating against weak attacks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5025–5034.
- Xie, C., Zhang, Z., Zhou, Y., Bai, S., Wang, J., Ren, Z., and Yuille, A. L. (2019). Improving transferability of adversarial examples with input diversity. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2730–2739.